Zadanie nr 1 na pracownię

Kolejki priorytetowe

Kolejką priorytetową nazywamy strukturę danych która, podobnie jak poznane na wykładzie kolejki FIFO, reprezentuje pewien zbiór elementów oraz umożliwia dodawanie i usuwanie elementów. Z każdym elementem kolejki priorytetowej jest skojarzona wartość nazywana *kluczem*. W przeciwieństwie do kolejek FIFO, w których kolejność usuwania elementów jest tożsama z kolejnością ich dodania, w kolejce priorytetowej kolejność ta jest zależna od wartości kluczy: jako pierwszy jest usuwany z kolejki element, który posiada najmniejszy klucz.

Implementacja kolejki priorytetowej w języku OCaml może spełniać poniższą sygnaturę:

```
type ('a, 'b) pq
val empty : ('a, 'b) pq
val insert : 'a -> 'b -> ('a, 'b) pq -> ('a, 'b) pq
val pop : ('a, 'b) pq -> ('a, 'b) pq
val min : ('a, 'b) pq -> 'b
val min_prio : ('a, 'b) pq -> 'a
```

Funkcje wymagające niepustej kolejki powinny, w przypadku otrzymania kolejki pustej, przerwać działanie przy użyciu `failwith`.

Podobnie jak kolejki FIFO, kolejki priorytetowe można zaimplementować na wiele sposobów. Spośród z nich jednym z najprostszych jest wykorzystanie listy posortowanej względem wartości klucza. Dostęp do pierwszego elementu jest wtedy bardzo efektywny (w czasie stałym), jednakże dodanie nowego elementu odbywa się w czasie liniowym. Inna, efektywniejsza implementacja, wykorzystuje strukturę danych znaną jako kopce lewicowe.

Kopce lewicowe

Kopce lewicowe (znane też jako drzewa lewicowe) to prosta i efektywna struktura danych implementująca kolejkę priorytetową zaproponowana w 1972 roku przez Clarka Crane'a i uproszczona rok później przez Donalda Knutha. Struktura ta umożliwia dostęp do najmniejszego elementu w czasie stałym oraz wstawianie i usuwanie elementów w czasie logarytmicznym. Zamiast listy budujemy *drzewo binarne*, w którym wierzchołki zawierają elementy kopca wraz z kluczami. Klucz w korzeniu każdego wierzchołka powinien być nie większy niż klucze w jego poddrzewach. Dodatkowym niezmiennikiem struktury danych, który umożliwi efektywną implementację jest to, że każdemu kopcowi przypisujemy *rangę*, którą jest długość „prawego kręgosłupa" (czyli ranga prawego poddrzewa zwiększona o 1 – lub zero w przypadku pustego kopca), i że w każdym poprawnie sformowanym kopcu ranga lewego poddrzewa jest nie mniejsza niż ranga prawego poddrzewa.

Przyjmijmy następującą definicję drzewa:

```
type ('a, 'b) heap = Leaf | Node of ('a, 'b) heap * int * 'a *
    'b * ('a, 'b) heap
```

Kolejne pola konstruktora `Node` oznaczają: lewe podrzewo, rangę, klucz, wartość, prawe poddrzewo. Rangą liścia jest 0:

```
let rank h =
  match h with
  | Leaf -> 0
  | Node (_, r, _, _, _) -> r
```

Aby mieć pewność, że konstruowane przez nas wartości typu `heap` spełniają niezmienniki kopca lewicowego, będziemy chcieli zaimplementować *inteligentny konstruktor* węzła:

```
val mk_node : ('a, 'b) heap * 'a * 'b * ('a, 'b) heap -> ('a,
    'b) heap
```

Zwróć uwagę, że nie przyjmuje on rangi tworzonego kopca, ale musi ją wyliczyć. W procedurze konstruktora należy też stwierdzić, który z kopców powinien zostać prawym, a który lewym poddrzewem (możemy natomiast założyć że porządek kopca zostanie zachowany).

Kluczową operacją na kopcach lewicowych jest scalanie dwóch kopców:

```
val merge : ('a, 'b) heap -> ('a, 'b) heap -> ('a, 'b) heap
```

Mając tę operację, łatwo już jest zaimplementować usunięcie elementu o najmniejszym kluczu (wystarczy scalić poddrzewa) oraz wstawienie nowego elementu (wystarczy scalić z kopcem jednoelementowym).

Idea scalania kopców jest następująca: jeśli jeden z kopców jest pusty, scalanie jest trywialne (bierzemy drugi kopiec). Jeśli oba są niepuste, możemy znaleźć najmniejszy element każdego z nich. Element o mniejszym kluczu powinien znaleźć się w korzeniu wynikowego kopca — łatwo go znaleźć. Mamy zatem cztery obiekty:

- element o najniższym priorytecie (nazwiemy go e),
- lewe poddrzewo kopca z którego korzenia pochodzi e — h_l
- prawe poddrzewo kopca z którego korzenia pochodzi e — h_r
- drugi kopiec, h, którego korzeń miał priorytet większy niż e.

Aby stworzyć wynikowy kopiec wystarczy teraz scalić h_r i h (rekurencyjnie), a następnie stworzyć wynikowy kopiec z kopca otrzymanego przez rekurencyjne scalanie, kopca h_l i elementu e. Poprawnie zdefiniowany konstruktor `mk_node` sam zadba o to, żeby otrzymany kopiec spełniał niezmienniki struktury danych.

Zadanie

Zaimplementuj kopce lewicowe, uzupełniając definicje w szablonie rozwiązania załączonym na SKOS. Zaimplementuj również algorytm *sortowania przez kopcowanie*, polegający na utworzeniu kopca zawierającego wszystkie elementy listy wejściowej, a następnie skonstruowanie listy wynikowej przez usuwanie elementów z kopca. Plik z rozwiązaniem, nazwany `solution.ml`, zgłoś przez system Web-CAT (dostęp przez odnośnik na SKOS) do dnia 8 kwietnia 2026, godz. 6:00.